



RAPPORT DE PROJET GDX2D : ASCENT

Kevin Ferreira

11 juin 2026

Table des matières

1 – Introduction	4
1.1 – Présentation du jeu	4
1.2 – Contrôles	4
1.3 – Éléments présents dans les niveaux	5
1.4 – Progression	5
1.5 – Conception des niveaux	5
1.6 – Organisation du rapport	6
2 – Problématique	6
2.1 – Objectifs et périmètre	6
3 – Choix d'implémentation	7
3.1 – Architecture générale	7
3.2 – Boucle de jeu et cycle de rendu	9
3.3 – Système de mouvement du joueur	10
3.4 – Détection et résolution des collisions	11
3.5 – Chargement et représentation des niveaux	12
3.6 – Constantes et réglage du gameplay	14
3.7 – Rendu graphique	14
3.8 – Écarts par rapport à la description initiale	15
4 – Difficultés rencontrées et solutions	15
4.1 – Inversion du système de coordonnées	15
4.2 – Collisions imprécises et tunneling	16
4.3 – Distinction murs / ressorts / pics	16
4.4 – Gestion de l'état du dash	16
4.5 – Chargement des fichiers de niveau	17
4.6 – Rendu graphique : centre vs coin	17
4.7 – Lignes de longueurs inégales dans les fichiers de niveau	18
4.8 – Réglage empirique des constantes de mouvement	18
4.9 – Difficultés organisationnelles	18
5 – Bilan : ce qui fonctionne et ce qui reste incomplet	19
5.1 – Fonctionnalités opérationnelles	19
5.2 – Fonctionnalités partiellement implémentées	20
5.3 – Limites connues	20
5.4 – Tests effectués	20
6 – Pistes d'amélioration et auto-critique	21
6.1 – Court terme	21
6.2 – Moyen terme	21
6.3 – Long terme	21
6.4 – Auto-critique sur le déroulement du projet	21
7 – Conclusion	22
8 – Références	23
Annexes	24

A.1. Main.scala	26
A.2. Ascent.scala	27
A.3. Player.scala	29
A.4. Level.scala	32
A.5. Wall.scala	33
A.6. Spike.scala	34
A.7. Spring.scala	35
A.8. Orb.scala	36
A.9. Exit.scala	37

1 – Introduction

Ce rapport présente Ascent, jeu de plateforme 2D développé dans le cadre du module 101.2 : Prog. orientée-objets. Le développement s'appuie sur la bibliothèque GDX2D, fournie par l'établissement et construite au-dessus de LibGDX. Elle permet de créer une fenêtre graphique, de gérer les entrées clavier et d'afficher des formes en temps réel, sans recourir à un moteur de jeu complet. Le langage utilisé pour ce projet est donc Scala.

Ayant toujours voulu créer un platformer, j'ai donc profité de l'occasion pour en réaliser un « prototype » en vue d'un futur projet personnel. Le jeu s'inspire de l'un de mes jeux préférés nommé Celeste. Un personnage y évolue dans des environnements verticaux et horizontaux, doit éviter des obstacles mortels et utiliser des mécaniques de mobilité, notamment le dash, pour atteindre la sortie de chaque niveau. Conformément aux consignes du projet, la fenêtre est fixée à 1920 × 1080 pixels.

Le projet a été réalisé seul. Le code source est hébergé sur GitHub à l'adresse suivante : <https://github.com/Sanko-kf/Ascent>.

1.1 – Présentation du jeu

Ascent est un jeu de plateforme en vue latérale dans lequel le joueur contrôle un personnage représenté par un rectangle cyan. L'objectif de chaque niveau est d'atteindre une sortie, affichée en jaune, sans toucher les pics, matérialisés par des rectangles rouges, qui provoquent un retour immédiat au point de départ.

Un chronomètre affiche le temps de la run en cours (*Time*) et le meilleur temps enregistré (*Best*). Après complétion du dernier niveau, le meilleur temps est sauvegardé avant de relancer une nouvelle partie.

1.2 – Contrôles

Le déplacement repose sur une physique à accélération et freinage progressifs, proche du ressenti de *Celeste*. Le dash constitue la mécanique centrale : il s'effectue dans huit directions et n'est utilisable qu'une fois par charge.

Touche	Action	Condition / remarque
← / →	Déplacement horizontal	Accélération et freinage progressifs
Espace	Saut	Uniquement lorsque le personnage est au sol
Shift gauche	Dash directionnel	Une fois par charge : direction selon les flèches maintenues (8 directions)
R	Relancer la run	Remet le chronomètre à zéro et recharge le premier niveau

Table 1 - Contrôles du joueur

Le dash se recharge lorsque le personnage touche le sol, utilise un ressort ou collecte une orbe.

1.3 – Éléments présents dans les niveaux

Outre le joueur, chaque niveau peut contenir les éléments suivants :

Élément	Représentation visuelle	Effet
Mur	Rectangle gris	Bloque le déplacement → délimite l'espace de jeu
Pic	Rectangle rouge	Mort instantanée → retour au point de départ
Sortie	Rectangle jaune	Déclenche le passage au niveau suivant
Orbe	Cercle vert	Recharge le dash → disparaît temporairement après utilisation
Ressort	Rectangle marron	Propulse le joueur vers le haut → recharge également le dash

Table 2 - Éléments interactifs des niveaux

1.4 – Progression

Les niveaux sont détectés automatiquement au démarrage par `discoverLevels()` : tous les fichiers `.txt` du dossier `levels/` sont chargés et triés par numéro. Lorsque le joueur atteint une sortie, le niveau suivant se charge automatiquement.

Après le dernier niveau, `finishRun()` compare le temps écoulé au meilleur temps enregistré, puis le jeu reprend au premier niveau. La touche `R` permet de relancer une run à tout moment depuis le début, avec réinitialisation du chronomètre.

1.5 – Conception des niveaux

Les niveaux ne sont pas codés en dur dans le programme : ils sont décrits dans des fichiers texte du dossier `levels/`, où chaque caractère correspond à une tuile de 48×48 pixels. Lors du chargement, le fichier est lu ligne par ligne : chaque symbole est converti en objet de jeu aux coordonnées appropriées.

Caractère	Élément associé
#	Mur
X	Pic
O	Orbe
S	Ressort
E	Sortie
P	Point de départ
.	Case vide

Table 3 - Correspondance des caractères ASCII

Cette approche sépare la conception des niveaux de la logique du jeu : l'ajout ou la modification d'un parcours se fait en éditant un simple fichier texte, sans devoir calculer pixel par pixel.

Ce choix répond à une priorité de conception : plutôt que de multiplier les parcours codés en dur, le projet mise sur un système modulaire. Le jeu en propose plusieurs par défaut, mais tout joueur peut composer sa propre run en ajoutant ou modifiant des fichiers `.txt` dans `levels/`, sans recompilation : `discoverLevels()` charge automatiquement tout nouveau fichier présent dans le dossier.

1.6 – Organisation du rapport

Le reste de ce document est structuré comme suit. Le chapitre 2 expose la problématique ainsi que les objectifs et le périmètre du projet. Le chapitre 3 documente les choix d'implémentation et les écarts éventuels par rapport à la description initiale du projet. Le chapitre 4 présente les difficultés rencontrées et les solutions adoptées. Le chapitre 5 dresse un bilan de ce qui fonctionne et de ce qui reste incomplet. Le chapitre 6 propose des pistes d'amélioration. Le chapitre 7 conclut, et le chapitre 8 liste les références utilisées. Le code source est reproduit en annexe.

2 – Problématique

Réaliser un jeu de plateforme fonctionnel avec GDX2D soulève plusieurs questions techniques qui ont dû être traitées :

Comment structurer le code ?

Un jeu interactif combine boucle de rendu, gestion des entrées, physique simplifiée et affichage. Il fallait définir quelles responsabilités confier à chaque classe, par exemple, une classe principale pour la boucle de jeu, une pour le joueur, une pour le chargement et l'affichage du niveau, et des classes distinctes pour chaque type d'objet (mur, pic, orbe, etc...).

Comment simuler le mouvement sans moteur physique ?

Les consignes recommandent d'éviter un moteur physique type Box2D sauf nécessité absolue. La gravité, l'accélération horizontale, le saut, le dash et la détection de collisions par boîtes englobantes ont donc été implémentés manuellement, avec une résolution séparée sur les axes horizontal et vertical.

Comment représenter et charger les niveaux ?

Il fallait choisir un format de données lisible et extensible. Le choix s'est porté sur des grilles de caractères ASCII, tout en gérant la conversion entre le système de coordonnées du fichier (ligne 0 en haut) et celui de GDX2D (origine en bas à gauche).

Comment articuler les mécaniques de jeu ?

Le dash, la recharge via les orbes, le rebond des ressorts et la mort sur les pics impliquent un état partagé entre le joueur et les objets du niveau. Il fallait définir clairement quand et comment ces interactions se produisent à chaque frame.

2.1 – Objectifs et périmètre

Les objectifs du projet sont les suivants :

- Mettre en place une architecture claire en Scala, avec un découpage cohérent entre la boucle de jeu (`Ascent`), le joueur (`Player`), le niveau (`Level`) et les entités interactives.
- Implémenter une physique de plateforme simplifiée : gravité, saut, dash directionnel et collisions avec les murs et ressorts.
- Concevoir un système de chargement de niveaux à partir de fichiers texte, avec une légende de caractères documentée.
- Intégrer les mécaniques inspirées de *Celeste* : dash à usage limité, orbes de recharge, ressorts, pics mortels et sortie de niveau.
- Proposer plusieurs niveaux jouables enchaînés, démontrant la viabilité du format de données choisi.

Le périmètre volontairement limité du projet reflète les priorités fixées par les enseignants : la note porte principalement sur le code et non sur le polish graphique ou sonore. Les éléments visuels sont donc minimalistes (rectangles et cercles colorés), ce qui a permis de concentrer les efforts sur la logique de jeu et le game feel.

La quantité de niveaux fournis reste volontairement modeste. L'effort a été concentré sur un pipeline de chargement extensible (`discoverLevels()`), afin de laisser au joueur la liberté d'enrichir ou de remplacer les parcours proposés et de constituer ses propres runs.

3 – Choix d'implémentation

Ce chapitre décrit l'architecture retenue pour Ascent, les principales décisions techniques et les écarts par rapport à la description initiale du projet. Il présente l'articulation entre les classes, l'organisation de la boucle de jeu, ainsi que les principes du mouvement du joueur, des collisions et du chargement des niveaux.

3.1 – Architecture générale

Le projet est structuré autour de huit classes et un objet Scala, regroupés dans le package `ch.hevs.gdx2d.game` :

- `Main` : point d'entrée du programme (objet singleton). Il instancie la classe principale du jeu.
- `Ascent` : classe centrale héritant de `PortableApplication(1920, 1080)`. Elle gère la boucle de jeu, le chargement des niveaux, l'enchaînement entre eux et l'affichage du chronomètre.
- `Player` : encapsule la position, la vitesse, l'état du joueur (au sol, en dash, charge disponible) ainsi que toute la logique de mouvement et d'interaction.
- `Level` : charge un fichier texte, convertit la grille en objets du monde et coordonne leur mise à jour et leur affichage.
- `Wall`, `Spike`, `Spring`, `Orb`, `Exit` : entités du niveau. Chacune est responsable de son rendu et de sa détection de collision.

Ce découpage suit le principe de responsabilité unique. La classe `Ascent` orchestre le déroulement global, `Player` concentre la logique du personnage, et `Level` agit comme conteneur des éléments statiques et interactifs du parcours. Les entités de niveau partagent une structure similaire (position, dimensions, méthodes `draw` et `hits`) sans toutefois passer par une interface ou un trait commun : un choix discuté plus bas.

Les dépendances entre ces classes se résument ainsi :

- `Main` : crée `Ascent`
- `Ascent` : possède `Player` et `Level`
- `Level` : contient des collections de `Wall`, `Spike`, `Spring`, `Orb`, `Exit`
- `Player` : interagit avec les collections exposées par `Level`

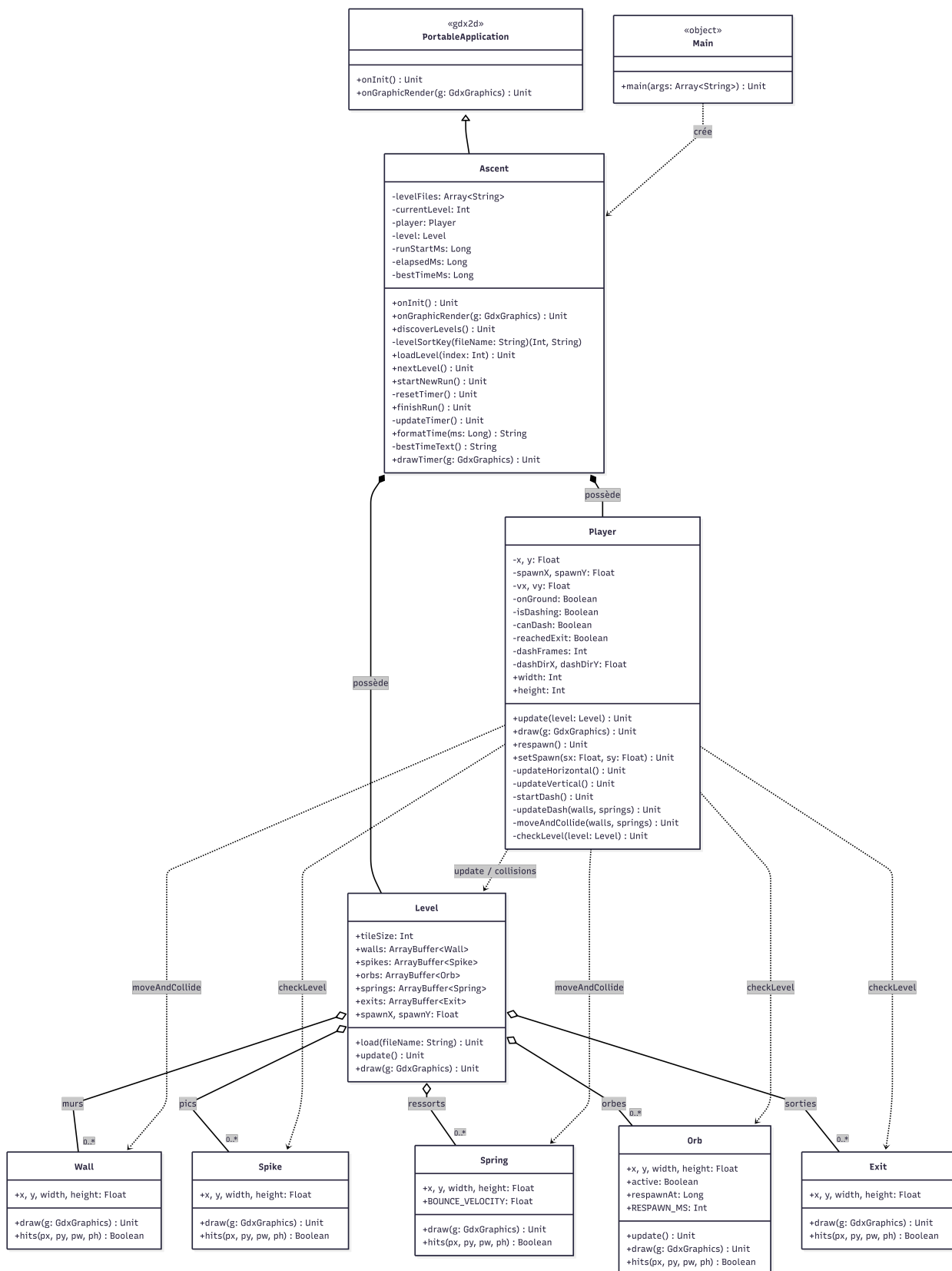


Figure 1 - Diagramme de classes

La Figure 1 met en évidence le fait que `Player` ne crée ni ne possède aucune entité : il obtient les murs, ressorts, pics, orbes et sorties via les `ArrayBuffer` exposés par `Level`, puis interagit directement avec ces instances (lecture des positions, déclenchement du rebond, désactivation d'une orbe).

3.2 – Boucle de jeu et cycle de rendu

Au démarrage, GDX2D appelle la méthode `onInit()` de la classe `Ascent`. Celle-ci définit le titre de la fenêtre (« Ascent »), découvre les niveaux disponibles via `discoverLevels()`, puis lance une nouvelle partie avec `startNewRun()`. Cette dernière méthode réinitialise le chronomètre et charge le premier niveau avec `loadLevel(0)`.

3.2.1 – Découverte des niveaux

Contrairement à un tableau de fichiers codé en dur, les niveaux sont désormais détectés automatiquement. La méthode `discoverLevels()` parcourt le dossier `levels/`, filtre les fichiers se terminant par `.txt`, puis les trie par numéro extrait du nom de fichier (par exemple `level1.txt` avant `level2.txt`). Le résultat est stocké dans le tableau `levelFiles`. Si le dossier est introuvable, le tableau reste vide : dans ce cas, aucun niveau ni joueur n'est instancié et la boucle de rendu échoue sur une référence nulle. Le programme ne gère donc pas gracieusement ce scénario, ce qui est recensé dans les limites connues (chapitre 5).

3.2.2 – Chargement d'un niveau

La méthode `loadLevel(index)` effectue les opérations suivantes :

1. vérifie que `levelFiles` n'est pas vide
2. met à jour l'index du niveau courant
3. instancie un nouvel objet `Level` à partir du nom de fichier correspondant
4. crée un nouveau `Player` positionné sur les coordonnées de spawn lues dans le fichier
5. enregistre ce point de spawn comme position de réapparition du joueur (`setSpawn`)

Le joueur est donc systématiquement réinitialisé à chaque changement de niveau. Cela évite de transporter un état de dash ou de vitesse d'un parcours à l'autre.

3.2.3 – Rendu périodique

À chaque image, `onGraphicRender(g: GdxGraphics)` exécute les opérations suivantes, dans cet ordre :

1. effacement de l'écran (`g.clear(Color.BLACK)`)
2. mise à jour du chronomètre (`updateTimer()`)
3. détection de la touche `R` pour relancer une partie (`startNewRun()`)
4. mise à jour des entités du niveau (`level.update()`) : principalement le respawn temporisé des orbes
5. affichage du niveau (`level.draw(g)`)
6. mise à jour du joueur (`player.update(level)`)
7. affichage du joueur (`player.draw(g)`)
8. affichage du chronomètre et du meilleur temps (`drawTimer(g)`)
9. si le joueur a atteint une sortie, passage au niveau suivant (`nextLevel()`)

La mise à jour du joueur précède son affichage, de sorte que les interactions (collisions, mort, collecte) sont toujours résolues avant le rendu du personnage. Le niveau étant dessiné avant la mise à jour du joueur, une orbe collectée pendant la frame reste visible une image de plus : ce décalage d'une frame est imperceptible en pratique.

3.2.4 – Enchaînement des niveaux

Lorsque le joueur atteint une sortie, `nextLevel()` calcule l'index suivant (avec retour à zéro après le dernier niveau) et le charge via `loadLevel`. Si le niveau qui vient d'être terminé était le dernier du dossier `levels/`, `finishRun()` est ensuite appelée : elle compare le temps écoulé au meilleur temps enregistré, le remplace si nécessaire, puis réinitialise le chronomètre. Le jeu recommence alors au premier niveau.

3.3 – Système de mouvement du joueur

Le joueur est modélisé par sa position (x , y), sa vitesse (v_x , v_y) et plusieurs indicateurs booléens :

- `onGround` : vrai lorsque le personnage repose sur un mur
- `isDashing` : vrai pendant la durée du dash
- `canDash` : indique si une charge de dash est disponible
- `reachedExit` : signalé en fin de frame si une sortie a été touchée

Des variables complémentaires gèrent le dash (`dashFrames` , `dashDirX` , `dashDirY`) ainsi que le point de réapparition (`spawnX` , `spawnY`).

Le personnage occupe une boîte de collision de 48×64 pixels, légèrement plus haute que large. Cela donne des proportions proches d'un personnage humain tout en restant aligné sur la grille de tuiles de 48 pixels.

3.3.1 – Déplacement horizontal

Le mouvement latéral repose sur une accélération constante (`ACCELERATION` = 2) plutôt que sur une vitesse fixe. Lorsque le joueur maintient une flèche directionnelle, v_x augmente ou diminue progressivement jusqu'à atteindre `MAX_SPEED` = 10 . Lorsqu'aucune touche n'est enfoncée, la même valeur d'accélération sert de frein jusqu'à ramener v_x à zéro. Ce comportement produit une sensation de glisse contrôlée, comparable aux platformers modernes, sans nécessiter de physique avancée.

Les entrées sont lues via l'API LibGDX : `Gdx.input.isKeyPressed` pour le maintien des flèches, `Gdx.input.isKeyJustPressed` pour le saut et le dash (actions ponctuelles).

3.3.2 – Gravité et saut

À chaque frame hors dash, la vitesse verticale est diminuée de `GRAVITY` = 0.667 , avec un plafond de chute `MAX_FALL` = 26.67 . L'axe Y de GDX2D pointant vers le haut, une vitesse verticale négative correspond à une chute.

Le saut n'est autorisé que si `onGround` est vrai. Dans ce cas, v_y est fixée à `JUMP_VELOCITY` = 15 et `onGround` repasse à faux. Cette condition empêche le double saut en l'air, sauf si le joueur utilise un ressort ou un dash pour regagner une surface.

3.3.3 – Dash directionnel

Le dash est déclenché par `Shift gauche` , à condition que `canDash` soit vrai et que le joueur ne soit pas déjà en train de dasher. La direction est déterminée par les flèches actuellement enfoncées au moment de l'appui (huit directions possibles). Si aucune direction n'est indiquée, le dash suit le sens du mouvement horizontal courant (droite par défaut si $v_x \geq 0$, gauche sinon).

Le vecteur directionnel est normalisé, puis appliqué pendant `DASH_DURATION` = 9 frames à la vitesse `DASH_SPEED` = 25 . Pendant cette phase, les entrées de mouvement classiques sont ignorées. Seules les collisions avec murs et ressorts sont prises en compte. À la fin du dash, les vitesses sont divisées par deux pour adoucir la transition vers le mouvement normal.

La charge de dash (`canDash`) est consommée au lancement et ne se recharge qu'en touchant le sol, un ressort ou une orbe.

3.4 – Détection et résolution des collisions

3.4.1 – Boîtes englobantes (AABB)

Toutes les entités du jeu utilisent une détection de collision par boîtes englobantes axis-aligned (AABB). Chaque objet expose une méthode `hits(px, py, pw, ph)` qui retourne vrai si le rectangle du joueur chevauche celui de l'objet :

$$px < x + width \ \&\& \ px + pw > x \ \&\& \ py < y + height \ \&\& \ py + ph > y$$

Cette formule, identique sur `Wall`, `Spike`, `Spring` et `Exit`, constitue le socle de toutes les interactions physiques du jeu. Pour les orbes, la méthode `hits` vérifie en plus que l'orbe est active (`active = true`).

3.4.2 – Résolution séparée des axes

La méthode `moveAndCollide` applique le déplacement en deux temps, conformément à une approche classique en jeux 2D sans moteur physique. Seuls les murs et les ressorts y participent.

1. Axe horizontal : la position `x` est mise à jour, puis testée contre chaque mur et chaque ressort. En cas de chevauchement, le joueur est repositionné au bord de l'obstacle et `vx` est annulée.
2. Axe vertical : `onGround` est remis à faux, puis `y` est mise à jour et testée de la même manière. Si le joueur descend sur un mur (`vy < 0`), il est placé au-dessus de la surface, `onGround` passe à vrai et `canDash` est rechargé. S'il monte dans un plafond (`vy > 0`), il est repoussé vers le bas.

Les ressorts se comportent comme des murs pour les collisions latérales et lorsque le joueur les heurte par en dessous. En revanche, lorsque le joueur atterrit dessus par le haut, ils déclenchent un rebond (`BOUNCE_VELOCITY = 24`) au lieu d'annuler la vitesse verticale comme le ferait un mur. Dans ce cas, `onGround` reste à faux car le joueur est immédiatement propulsé vers le haut, mais `canDash` est tout de même rechargé.

3.4.3 – Interactions non physiques

Après le déplacement, la méthode `checkLevel` parcourt les pics, orbes et sorties :

- Pics : tout contact appelle `respawn()`, qui replace le joueur au point de spawn et réinitialise vitesse et état de dash
- Orbes : le contact recharge le dash, désactive l'orbe et programme sa réapparition après 3 secondes (`RESPAWN_MS = 3000`), gérée dans `Orb.update()` via `System.currentTimeMillis()`
- Sorties : le contact positionne `reachedExit` à vrai, ce qui déclenche le changement de niveau dans `Ascent`

Les pics ne bloquent pas physiquement le joueur. Ils tuent sans agir comme un mur. Ce choix simplifie le level design (les pics peuvent être traversés une frame avant le `respawn`) et évite de coincer le joueur dans une zone mortelle.

3.5 – Chargement et représentation des niveaux

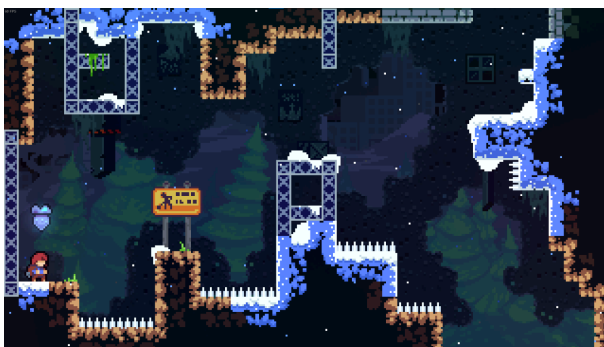
Chaque niveau est un fichier texte ASCII dans le dossier `levels/`. Chaque ligne du fichier correspond à une rangée de tuiles, lue de haut en bas. Chaque caractère représente une case de la grille. La légende retenue est la suivante :

Symbole	Élément	Couleur d’affichage
#	Mur	Gris
X	Pic mortel	Rouge
S	Ressort	Marron
O	Orbe de recharge	Vert (cercle)
E	Sortie de niveau	Jaune
P	Point de spawn du joueur	aucune
.	Case vide	aucune

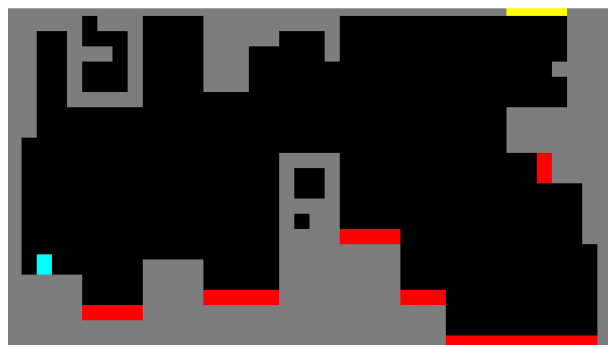
Table 4 - Légende des caractères ASCII

Les cases vides sont ignorées par le parseur (case `_` $\Rightarrow$). Le caractère `P` ne crée pas d’objet visible. Il enregistre uniquement les coordonnées de spawn dans `spawnX` et `spawnY`.

Comme dans *Celeste*, *Ascent* reprend un level design vertical : le joueur progresse en sautant et en dashant entre plateformes, en évitant les dangers pour atteindre la sortie. La Figure 2 compare le niveau d’origine et sa reprise dans le prototype.



(a) Niveau de Celeste (référence)



(b) Même niveau recréé dans Ascent

Figure 2 - Inspiration *Celeste* (a) et reprise dans *Ascent* (b)

Sous le capot, *Celeste* ne dessine pas son monde librement : chaque élément est placé sur une grille de tuiles. Murs, pics, plateformes et objets interactifs occupent des cases régulières, ce qui simplifie les collisions et la conception des parcours. La Figure 3 illustre ce principe.

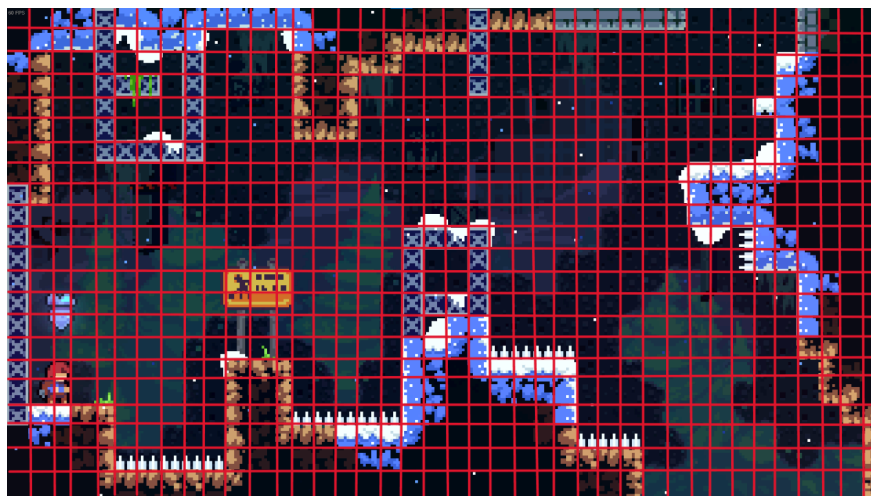


Figure 3 - Le niveau de Celeste repose sur une grille de tuiles

Ascent applique la même logique, mais sans éditeur graphique ni tileset. Chaque case de la grille est décrite par un caractère ASCII dans un fichier texte du dossier `levels/`. Lors du chargement, le parseur lit le fichier ligne par ligne et instancie les objets correspondants (Wall, Spike, Spring, etc.). La Figure 4 montre cette correspondance pour le premier niveau.

```
#####EEEE#####
####.###.....#####.#####
##.#...#...#####...#.....#####
##.###.#...###.....#.....#####
##.#...#...###.....#####
##.#...#...###.....#####
##.#####.....#####
##.....#####
##.....#####
#.....#####
#.....####.....X#####
#.....#.#.....X#####
#.....#.#.....####
#.....####.....####
#.....#.#.....####
#.....####XXXX.....####
#.....#####.....###
#.P.....####.....#####
#####.....#####
#####.....XXXXX#####XXX.....
#####XXXX#####.....
#####.....
#####XXXXXXXXX###
```

Figure 4 - Correspondance entre le fichier ASCII et le premier niveau

Le joueur cyan (P) part du point de spawn en bas à gauche. Les murs gris (#), les pics rouges (x) et la sortie jaune (E) en haut à droite correspondent directement aux symboles du fichier.

3.5.1 – Conversion des coordonnées

GDX2D place l'origine en bas à gauche de l'écran, tandis que la première ligne d'un fichier texte se trouve en haut de la grille. L'axe Y croît donc vers le haut dans le moteur, alors que la lecture du fichier progresse de haut en bas.

Lors du chargement, chaque tuile située à la ligne `row` et à la colonne `col` est convertie en coordonnées monde par :

```
worldX = col * tileSize
worldY = (nbLignes - 1 - row) * tileSize
```

avec `tileSize = 48` et `nbLignes` le nombre total de lignes du fichier.

La coordonnée `worldX` place la tuile horizontalement en fonction de sa colonne. La formule de `worldY` inverse l'axe vertical pour aligner la première ligne du fichier sur la rangée de tuiles la plus haute à l'écran. Sans cette inversion, le niveau apparaît retourné : le spawn et la sortie se retrouvent aux positions opposées à celles prévues dans le fichier ASCII.

Le point de spawn `P` et chaque symbole (`#`, `X`, `E`, etc.) sont donc positionnés dans le même repère que le joueur et les collisions, ce qui garantit la cohérence entre le fichier texte, l'affichage et la physique du jeu.

3.6 – Constantes et réglage du gameplay

Les constantes de mouvement sont déclarées directement dans la classe `Player` sous forme de `val` :

Constante	Valeur	Rôle
MAX_SPEED	10	Vitesse horizontale maximale
ACCELERATION	2	Accélération et freinage
GRAVITY	0.667	Accélération de chute par frame
MAX_FALL	26.67	Vitesse de chute maximale
JUMP_VELOCITY	15	Impulsion du saut
DASH_SPEED	25	Vitesse pendant le dash
DASH_DURATION	9	Durée du dash en frames

Table 5 - Constantes de mouvement du joueur

Ces valeurs ont été ajustées empiriquement en jouant aux niveaux, plutôt que calculées à partir d'une simulation physique réaliste. Cette approche de réglage manuel est courante en développement de jeux indépendants. Elle a permis d'obtenir un ressenti proche de *Celeste* sans moteur physique.

3.7 – Rendu graphique

Faute de sprites ou de tilesets, chaque entité se dessine via les primitives de `GdxGraphics` :

- rectangles remplis (`drawFilledRectangle`) pour murs, pics, ressorts, sorties et joueur
- cercle rempli (`drawFilledCircle`, rayon 14) pour les orbes
- texte (`drawString`) pour l'affichage du chronomètre et du meilleur temps

Les coordonnées passées à `drawFilledRectangle` correspondent au *centre* de la forme, d'où l'usage systématique de `x + width / 2` et `y + height / 2` dans chaque méthode `draw`. Le fond est noir uni. Aucun défilement de caméra n'est implémenté. L'intégralité du niveau est visible dans la fenêtre 1920 × 1080, ce qui limite la taille des grilles utilisables.

3.8 – Écarts par rapport à la description initiale

Lors de la réflexion du projet à ses tout début, la description retenue visait un platformer inspiré de *Celeste* avec dash, orbes et plusieurs niveaux. L'implémentation finale respecte globalement cette vision, mais présente les écarts suivants :

Prévu initialement	Implémenté	Commentaire
Écran de fin après le dernier niveau	Retour automatique au premier niveau	<code>finishRun()</code> enregistre le meilleur temps avant de relancer
Liste de niveaux fixe dans le code	Découverte automatique via <code>discoverLevels()</code>	Écart positif. Tout fichier <code>.txt</code> dans <code>levels/</code> est chargé et trié par numéro
Interface commune pour les entités	Classes indépendantes avec méthodes dupliquées	Choix pragmatique pour gagner du temps. Une refactorisation vers un trait <code>Collidable</code> reste envisageable
Graphismes soignés	Formes géométriques colorées	Priorité donnée au code, conformément aux consignes
Suppression des traces de débogage	<code>println</code> conservés	Conservés provisoirement comme traces des événements du jeu (dash, mort, chargement).
Mechanic avancée	aucune implémentée	Plus compliqué que prévu notamment pour le climbing et le speed dash

Table 6 - Écarts entre la description initiale et l'implémentation finale

Ces écarts ne remettent pas en cause le fonctionnement du jeu, mais certains devront être traités ou justifiés explicitement dans le bilan (chapitre 5).

4 – Difficultés rencontrées et solutions

Au cours du développement d'Ascent, plusieurs obstacles techniques et organisationnels se sont présentés. Ce chapitre retrace les principaux problèmes rencontrés, la démarche adoptée pour les diagnostiquer, et les solutions retenues, y compris les compromis acceptés.

4.1 – Inversion du système de coordonnées

4.1.1 – Symptôme

Lors des premiers tests de chargement de niveau, la disposition affichée à l'écran ne correspondait pas au fichier texte. Ce qui devait apparaître en haut du niveau se retrouvait en bas, et inversement. Le point de spawn du joueur (P) était parfois placé dans une zone incohérente par rapport aux murs dessinés.

4.1.2 – Cause

Les fichiers de niveau sont lus ligne par ligne, la première ligne représentant visuellement le haut de la grille. Or, dans GDX2D, l'origine du repère se situe en bas à gauche de la fenêtre. L'axe Y croît vers le haut. Une conversion directe `worldY = row * tileSize` provoquait donc un miroir vertical du niveau.

4.1.3 – Solution

Une inversion explicite a été introduite lors du parsing :

```
worldY = (nbLignes - 1 - row) * tileSize
```

où `nbLignes` est le nombre total de lignes du fichier. Cette formule aligne la première ligne du fichier sur la rangée de tuiles la plus haute à l'écran. Une fois ce correctif appliqué, les niveaux se sont affichés conformément à leur représentation ASCII.

4.2 – Collisions imprécises et tunneling

4.2.1 – Symptôme

Dans les premières versions du mouvement, le joueur pouvait traverser les murs à grande vitesse, notamment pendant le dash, ou rester coincé dans une plateforme après un atterrissage. Les collisions latérales et verticales se perturbaient mutuellement lorsqu'elles étaient traitées en une seule passe.

4.2.2 – Cause

Une résolution simultanée des deux axes ne permet pas de déterminer correctement si le joueur arrive par le côté ou par le dessus d'un mur. De plus, appliquer le déplacement complet $(x + vx, y + vy)$ avant de tester les collisions augmente le risque de franchir un obstacle fin en une seule frame (tunneling).

4.2.3 – Solution

Une résolution séparée des axes a été adoptée dans `moveAndCollide` :

1. déplacement et test sur l'axe X, puis annulation de `vx` en cas de contact
2. remise à zéro de `onGround`, puis déplacement et test sur l'axe Y
3. traitement distinct des murs et des ressorts sur chaque axe

Cette approche, bien que simpliste par rapport à un moteur physique, s'est révélée suffisante pour les niveaux du projet. Le dash, plus rapide que la marche normale, reste sensible aux angles très aigus entre murs, mais ne traverse plus systématiquement les obstacles.

4.3 – Distinction murs / ressorts / pics

4.3.1 – Symptôme

Les ressorts devaient propulser le joueur vers le haut, les murs devaient l'arrêter net, et les pics devaient le tuer sans le bloquer. Une gestion uniforme de tous les obstacles aurait empêché ces comportements différenciés.

4.3.2 – Cause

Les classes `Wall`, `Spring` et `Spike` partagent la même structure de collision AABB, ce qui tentait de les fusionner dans une seule boucle de traitement. Or leurs réactions diffèrent. Un mur annule la vitesse, un ressort applique `BOUNCE_VELOCITY`, un pic déclenche un respawn sans correction de position.

4.3.3 – Solution

Des classes séparées ont été conservées et la logique a été scindée en deux phases :

- Collisions physiques (`moveAndCollide`) : murs et ressorts uniquement, avec comportements distincts sur l'axe vertical
- Interactions logiques (`checkLevel`) : pics, orbes et sorties, testés après le déplacement

Les pics ne modifient pas la position du joueur. `respawn()` remplace directement le joueur au spawn. Ce choix évite qu'un pic agisse comme un mur invisible et simplifie la conception des passages dangereux.

4.4 – Gestion de l'état du dash

4.4.1 – Symptôme

À certains moments, le joueur pouvait enchaîner plusieurs dashes sans toucher le sol, ou perdre définitivement sa charge après un dash raté. La recharge via les orbes ne se déclenchait pas de manière fiable.

4.4.2 – Cause

L'état du dash repose sur trois variables interdépendantes : `isDashing`, `canDash` et `dashFrames`. Sans règles claires sur le moment de recharger `canDash`, le comportement devenait imprévisible. De plus,

pendant le dash, les entrées de mouvement classiques continuaient d'être lues dans les premières versions.

4.4.3 – Solution

Des règles explicites ont été définies :

- `canDash` passe à `false` au lancement du dash
- `canDash` repasse à `true` lorsque le joueur atterrit sur un mur (`onGround`)
- `canDash` repasse également à `true` lors d'un rebond sur un ressort ou de la collecte d'une orbe
- pendant `isDashing`, seule `updateDash` est appelée, ignorant `updateHorizontal` et `updateVertical`

À la fin du dash, les vitesses sont divisées par deux pour éviter une transition brutale. Le respawn sur pic réinitialise intégralement l'état du dash.

4.5 – Chargement des fichiers de niveau

4.5.1 – Symptôme

Lors de l'exécution depuis IntelliJ, le jeu plantait ou chargeait un niveau vide, avec une erreur liée à l'absence du fichier `levels/level1.txt`.

4.5.2 – Cause

Le chemin `levels/$fileName` est relatif au répertoire de travail courant au moment du lancement, et non au fichier source Scala. Si le programme est exécuté depuis un autre dossier, `Source.fromFile` ne trouve pas les fichiers.

4.5.3 – Solution

Le répertoire de travail de la configuration d'exécution IntelliJ a été fixé sur la racine du projet, où se trouve le dossier `levels/`. Cette solution reste fragile. Un lancement depuis la ligne de commande ou un export JAR Ant nécessite de vérifier que les fichiers sont bien copiés à côté de l'exécutable.

Par la suite, `discoverLevels()` a été ajouté pour détecter automatiquement tous les fichiers `.txt` du dossier, sans les coder en dur dans le source. Le problème du répertoire de travail demeure toutefois inchangé.

4.6 – Rendu graphique : centre vs coin

4.6.1 – Symptôme

Au début du développement, les tuiles et le joueur apparaissaient décalés par rapport à la grille attendue. Les formes semblaient flotter entre les cases plutôt que s'y aligner.

4.6.2 – Cause

La méthode `drawFilledRectangle` de GDX2D attend les coordonnées du centre du rectangle, et non du coin inférieur gauche. Les positions `x` et `y` modélisent pourtant le coin inférieur gauche de chaque tuile.

4.6.3 – Solution

Chaque appel de dessin utilise désormais :

```
g.drawFilledRectangle(x + width / 2, y + height / 2, width, height, ...)
```

La détection de collision, elle, continue d'utiliser le coin inférieur gauche. Cela maintient la cohérence entre la logique physique et l'affichage.

4.7 – Lignes de longueurs inégales dans les fichiers de niveau

4.7.1 – Symptôme

Certaines lignes du fichier `level2.txt` contiennent des points (.) en fin de ligne, produisant des largeurs de grille variables selon les rangées.

4.7.2 – Cause

Les fichiers ont été édités manuellement sans contrainte de largeur uniforme. Le parseur itère de 0 à `line.length`, ce qui crée des colonnes supplémentaires pour les lignes les plus longues.

4.7.3 – Solution

Ce comportement a été accepté. Les cases vides (.) sont ignorées et n'affectent pas le gameplay. Pour les niveaux futurs, une validation de largeur constante serait souhaitable, mais cela n'a pas bloqué le fonctionnement des niveaux actuels.

4.8 – Réglage empirique des constantes de mouvement

4.8.1 – Symptôme

Les premières valeurs de gravité, de saut et de dash produisaient un ressenti insatisfaisant : tantôt un personnage trop lourd et mou, tantôt un dash incontrôlable qui projetait le joueur hors des plateformes. Aucun jeu de valeurs choisi « au jugé » ne donnait d'emblée la nervosité recherchée.

4.8.2 – Cause

Sans moteur physique, les constantes de `Player` ne peuvent pas être dérivées d'une simulation réaliste. Surtout, elles sont couplées : modifier `GRAVITY` change la hauteur de saut utile, ce qui force à réajuster `JUMP_VELOCITY`, lequel influe à son tour sur la portée du dash. Cette interdépendance rend le résultat difficile à anticiper sur le papier.

4.8.3 – Solution

Le réglage s'est fait par itérations courtes : modifier une constante, relancer, rejouer un passage type, comparer. Plusieurs cycles ont été nécessaires pour stabiliser l'ensemble.

4.9 – Difficultés organisationnelles

Au-delà des aspects techniques, le déroulement du projet en autonomie a posé quelques défis :

- Priorisation : plutôt que de concevoir un grand nombre de salles, le développement a privilégié un système de niveaux modulaire (fichiers ASCII et `discoverLevels()`), permettant au joueur de constituer ses propres runs. La tentation d'ajouter des graphismes ou des sons a également été écartée au profit du code, conformément aux consignes
- Démarrage tardif : une partie du développement a dû être réalisée dans un délai restreint, ce qui a imposé de traiter en priorité les mécaniques essentielles (mouvement, dash, collisions) avant les fonctions secondaires (chronomètre, découverte automatique des niveaux)
- Tests sur l'environnement d'exécution : le jeu pouvait fonctionner ou échouer selon le répertoire de travail configuré dans IntelliJ, ce qui a retardé le diagnostic des erreurs de chargement

Les sessions de développement concentrées ont permis de résoudre les problèmes de collision, de valider le ressenti du dash et de tester les orbes de recharge sur les parcours disponibles.

5 – Bilan : ce qui fonctionne et ce qui reste incomplet

Ce chapitre dresse un état des lieux honnête du projet au moment du rendu, en distinguant les fonctionnalités opérationnelles des limites connues.

5.1 – Fonctionnalités opérationnelles

Les éléments suivants sont implémentés et testés sur les niveaux disponibles :

Fonctionnalité	État
Fenêtre fixe 1920 × 1080, mode fenêtre	Fonctionnel
Boucle de jeu GDX2D (onInit, onGraphicRender)	Fonctionnel
Déplacement horizontal avec accélération et freinage	Fonctionnel
Gravité, saut (au sol uniquement)	Fonctionnel
Dash directionnel avec charge limitée	Fonctionnel
Collisions AABB murs / ressorts	Fonctionnel
Morts sur pics avec respawn au spawn	Fonctionnel
Ressorts avec rebond vertical	Fonctionnel
Orbes de recharge (collecte, respawn temporisé)	Fonctionnel
Sorties de niveau et enchaînement automatique	Fonctionnel
Chargement de niveaux depuis fichiers ASCII	Fonctionnel
Découverte automatique des niveaux (discoverLevels)	Fonctionnel
Plusieurs niveaux jouables de difficulté croissante	Fonctionnel
Chronomètre et meilleur temps affichés à l'écran	Fonctionnel
Relance d'une partie avec la touche R	Fonctionnel

Table 7 - Fonctionnalités opérationnelles

Le gameplay de base est pleinement jouable : déplacement, saut, dash, collecte d'orbes, mort, rebond et progression. Le format de fichiers texte permet de modifier les parcours sans toucher au code source.

5.2 – Fonctionnalités partiellement implémentées

Fonctionnalité	État
Fin de partie	Après le dernier niveau, retour au premier avec enregistrement du meilleur temps. Pas d'écran de victoire dédié
Chronomètre	Temps global de la run et meilleur temps fonctionnels, mais pas de pause à la fin d'une run ni de temps par niveau
Robustesse du chargement	Les niveaux valides se chargent correctement, mais aucune validation des fichiers : dossier <code>levels/</code> manquant, niveau sans spawn <code>P</code> ou sans sortie <code>E</code> ne produisent pas de message d'erreur explicite
Retour visuel de l'état du dash	La charge de dash est gérée en interne (<code>canDash</code>), mais aucun indicateur à l'écran ne permet au joueur de savoir si elle est disponible

Table 8 - Fonctionnalités partiellement implémentées

5.3 – Limites connues

- Collisions simplistes : pas de gestion des pentes, des coins arrondis ni du tunneling à très haute vitesse sur murs fins.
- Traces de débogage : des appels `println` subsistent dans `Player`, `Level` et `Ascent` ("dash", "mort", "orbe", "ressort", nombre de murs chargés, liste des niveaux trouvés).
- Pas de tests automatisés : la validation repose exclusivement sur des tests manuels en jeu.
- Chemin relatif `levels/` : le chargement dépend du répertoire de travail au lancement.
- Duplication de code : la méthode `hits` est quasi identique sur `Wall`, `Spike`, `Spring` et `Exit`. Un trait commun aurait réduit la redondance.
- Graphismes et audio : aucun sprite, tileset ni son n'a été intégré. Le rendu repose sur des primitives géométriques colorées.
- Dossier `levels/` manquant non géré : si le dossier est absent au lancement, aucun niveau n'est chargé et le programme échoue sur une référence nulle au lieu d'afficher un message d'erreur.

5.4 – Tests effectués

Les tests manuels ont couvert les scénarios suivants :

1. parcours complet de tous les niveaux jusqu'à la sortie
2. mort sur pic et vérification du respawn au point `P`
3. utilisation du dash dans les huit directions
4. rebond sur ressort
5. collecte d'une orbe et recharge du dash
6. recharge du dash au sol après un saut
7. retour au niveau 1 après complétion du dernier niveau
8. affichage du chronomètre et enregistrement du meilleur temps
9. relance d'une partie avec la touche `R`

Aucun test automatisé n'a été écrit. Les régressions sont détectées en rejouant les niveaux après chaque modification significative.

6 – Pistes d'amélioration et auto-critique

6.1 – Court terme

- Nettoyer les `println` de débogage dans `Player`, `Level` et `Ascent`
- Implémenter un écran de fin simple après le dernier niveau (texte affiché via `GDX2D`, arrêt de la boucle ou retour menu)

6.2 – Moyen terme

- Implémenter un menu principal et un écran de pause
- Ajouter un HUD affichant l'état de la charge de dash
- Valider la largeur uniforme des fichiers de niveau à la lecture, avec message d'erreur explicite
- Améliorer le chronomètre : pause automatique à la fin, temps par niveau, affichage plus lisible

6.3 – Long terme

- Éditeur de niveaux intégré ou externe avec export ASCII
- Intégration de sprites et de tilesets
- Sons de saut, dash, mort et collecte d'orbe
- Compteur de morts et ghost replay à la manière de *Celeste*
- Mécaniques additionnelles : strawberries, cassettes, variantes de dash

6.4 – Auto-critique sur le déroulement du projet

6.4.1 – Points positifs

- Le découpage en classes a été pensé tôt, ce qui a limité les refactorisations majeures en fin de projet
- Le format ASCII pour les niveaux s'est révélé efficace : un parcours peut être conçu dans un éditeur de texte pendant que la logique de mouvement est développée en parallèle
- Le choix de s'inspirer de *Celeste* a fourni un cadre clair pour les mécaniques à implémenter, évitant une dispersion sur trop de fonctionnalités
- La lecture des démos `GDX2D` en début de projet a accéléré la prise en main de la bibliothèque
- L'ajout tardif du chronomètre et de `discoverLevels()` montre une capacité à itérer sans remettre en cause l'architecture existante

6.4.2 – Points à améliorer

- Estimation du temps : la partie collisions et dash a pris plus de temps que prévu. La conception des niveaux a été repoussée en conséquence
- Tests plus tôt : du temps aurait été gagné en testant le chargement des niveaux et les coordonnées `Y` dès la première semaine de code, plutôt qu'après avoir implémenté le mouvement complet
- Documentation au fil de l'eau : le rapport a été rédigé en fin de projet. Une documentation continue des choix techniques aurait facilité la rédaction
- Versionnement : un dépôt Git avec des commits réguliers aurait mieux tracé l'évolution du code et facilité les retours en arrière

6.4.3 – Organisation du travail en autonomie

Le projet a été mené seul, ce qui a simplifié les décisions d'architecture mais concentré l'ensemble des tâches sur une seule personne :

Domaine	Réalisation
Architecture et classe <code>Ascent</code>	Boucle de jeu, chronomètre, découverte des niveaux
Classe <code>Player</code> (mouvement, dash, collisions)	Physique du personnage et interactions
Classe <code>Level</code> et entités (<code>Wall</code> , <code>Spike</code> , etc.)	Chargement ASCII et objets du niveau
Conception des fichiers de niveaux	Plusieurs parcours inspirés de <i>Celeste</i>
Tests, rapport et médias de rendu	Validation manuelle et documentation

Table 9 - Répartition des tâches en projet solo

L'interdépendance entre `Player` et `Level` a nécessité de travailler sur les deux en alternance plutôt que de les isoler complètement. Le format ASCII a toutefois permis de découpler partiellement la conception des niveaux de la logique de jeu.

7 – Conclusion

Ascent est un jeu de plateforme 2D développé en Scala avec GDX2D, dans lequel le joueur progresse à travers plusieurs niveaux en utilisant saut, dash, orbes et ressorts pour atteindre des sorties tout en évitant les pics. Ce projet a permis de passer d'exercices isolés à une application interactive complète, structurée en classes coopérantes et animée par une boucle de jeu en temps réel.

Sur le plan technique, les apprentissages principaux portent sur la modélisation orientée objet (séparation joueur, niveau et entités), l'implémentation manuelle d'une physique de plateforme simplifiée (gravité, AABB, résolution séparée des axes), le parsing de fichiers de données externes, et l'intégration avec une bibliothèque graphique (`PortableApplication`, `GdxGraphics`, entrées clavier `LibGDX`).

Le résultat atteint les objectifs fonctionnels fixés au départ : un jeu jouable, extensible par ajout de fichiers `.txt` dans le dossier `levels/`, où chaque joueur peut composer sa propre run sans modifier le code source, respectant les contraintes du cahier des charges (fenêtre 1920×1080 , pas de moteur physique externe). Les mécaniques prévues (dash, orbes, ressorts, pics et enchaînement des niveaux) sont implémentées et fonctionnelles. Il reste un écran de fin dédié et un rendu plus abouti à prévoir : ces lacunes sont connues et documentées.

Le développement en autonomie a également mis en évidence l'importance de la priorisation (code avant polish graphique), des tests précoces (chargement des niveaux, coordonnées, collisions) et d'une documentation régulière des choix techniques. Ces aspects organisationnels ont été aussi déterminants que les difficultés purement liées à la programmation.

En synthèse, *Ascent* constitue une base solide pouvant évoluer vers un platformer plus complet. Les mécaniques fondamentales sont en place. Les améliorations identifiées au chapitre 6 (chronomètre affiné, menu, sprites, audio) s'appuient sur une architecture suffisamment claire pour être ajoutées sans remise en cause complète du code existant.

8 – Références

- Dépôt source Ascent : <https://github.com/Sanko-kf/Ascent>
- Documentation GDX2D : <https://isc-hei.github.io/gdx2d/index.html>
- Documentation LibGDX (API sous-jacente, entrées clavier) : <https://libgdx.com/wiki/>
- Projet de démarrage `gdx2d-helloDesktop` et consignes du module 101.2 Prog. orientée-objets
- *Celeste* (Maddy Makes Games, 2018) : inspiration pour le dash, les orbes et le level design vertical
- M. Thorson, « Celeste & Forgiveness » (https://maddymakesgames.com/articles/celeste_and_forgiveness/index.html) : techniques de game feel (fenêtres de tolérance, indulgence du joueur)
- M. Thorson, GDC 2017, « Level Design Workshop : Designing Celeste » (<https://www.youtube.com/watch?v=4RlpMhBKNr0>) : conception des salles, grille de tuiles et itération sur les parcours

Annexes

Table des figures

.....	1
Figure 1 Diagramme de classes	8
Figure 2 Inspiration <i>Celeste</i> (a) et reprise dans <i>Ascent</i> (b)	12
Figure 3 Le niveau de <i>Celeste</i> repose sur une grille de tuiles	13
Figure 4 Correspondance entre le fichier ASCII et le premier niveau	13

A.1. Main.scala

```
package ch.hevs.gdx2d.game

object Main {

  def main(args: Array[String]): Unit = {
    new Ascent
  }
}
```

A.2. Ascent.scala

```
package ch.hevs.gdx2d.game

import ch.hevs.gdx2d.desktop.PortableApplication
import ch.hevs.gdx2d.lib.GdxGraphics
import com.badlogic.gdx.Gdx
import com.badlogic.gdx.Input
import com.badlogic.gdx.graphics.Color

import java.io.File

class Ascent extends PortableApplication(1920, 1080) {

  var levelFiles: Array[String] = Array.empty
  var currentLevel = 0

  var player: Player = null
  var level: Level = null

  var runStartMs: Long = 0
  var elapsedMs: Long = 0
  var bestTimeMs: Long = 0

  override def onInit(): Unit = {
    setTitle("Ascent")
    discoverLevels()
    startNewRun()
  }

  def discoverLevels(): Unit = {
    val dir = new File("levels")
    if (!dir.exists || !dir.isDirectory) {
      println("dossier levels introuvable")
      levelFiles = Array.empty
      return
    }

    levelFiles = dir.listFiles()
      .filter(f => f.isFile && f.getName.toLowerCase.endsWith(".txt"))
      .map(_.getName)
      .sortBy(levelSortKey)
      .toArray

    println("niveaux trouves : " + levelFiles.mkString(", "))
  }

  def levelSortKey(fileName: String): (Int, String) = {
    val number = ""\d+"".r.findFirstIn(fileName).map(_.toInt).getOrElse(Int.MaxValue)
    (number, fileName)
  }

  def resetTimer(): Unit = {
    runStartMs = System.currentTimeMillis()
    elapsedMs = 0
  }

  def startNewRun(): Unit = {
    resetTimer()
    loadLevel(0)
  }

  def loadLevel(index: Int): Unit = {
    if (levelFiles.isEmpty) return

    currentLevel = index
    level = new Level(levelFiles(index))
    player = new Player(level.spawnX, level.spawnY)
    player.setSpawn(level.spawnX, level.spawnY)
  }

  def nextLevel(): Unit = {
    val finishedRun = currentLevel == levelFiles.length - 1
    var next = currentLevel + 1
    if (next >= levelFiles.length) next = 0
    loadLevel(next)
    if (finishedRun) finishRun()
  }

  def finishRun(): Unit = {
    if (bestTimeMs == 0 || elapsedMs < bestTimeMs) {

```

```

        bestTimeMs = elapsedMs
    }
    resetTimer()
}

def updateTimer(): Unit = {
    elapsedMs = System.currentTimeMillis() - runStartMs
}

def formatTime(ms: Long): String = {
    val minutes = ms / 60000
    val seconds = (ms % 60000) / 1000
    val millis = ms % 1000
    f"$minutes%02d:$seconds%02d.$millis%03d"
}

def bestTimeText(): String = {
    if (bestTimeMs == 0) "--:--.---" else formatTime(bestTimeMs)
}

def drawTimer(g: GdxGraphics): Unit = {
    g.drawString(5, 15, s"Time: ${formatTime(elapsedMs)}")
    g.drawString(5, 35, s"Best: ${bestTimeText()}")
}

override def onGraphicRender(g: GdxGraphics): Unit = {
    g.clear(Color.BLACK)
    updateTimer()

    if (Gdx.input.isKeyJustPressed(Input.Keys.R)) {
        startNewRun()
    }

    level.update()
    level.draw(g)
    player.update(level)
    player.draw(g)
    drawTimer(g)

    if (player.reachedExit) {
        nextLevel()
    }
}
}

```

A.3. Player.scala

```
package ch.hevs.gdx2d.game

import ch.hevs.gdx2d.lib.GdxGraphics
import com.badlogic.gdx.graphics.Color
import com.badlogic.gdx.Gdx
import com.badlogic.gdx.Input
import scala.collection.mutable.ArrayBuffer

class Player(startX: Float, startY: Float) {
  var x: Float = startX
  var y: Float = startY

  var spawnX: Float = startX
  var spawnY: Float = startY

  var vx: Float = 0f
  var vy: Float = 0f

  var onGround: Boolean = false
  var isDashing: Boolean = false
  var canDash: Boolean = true

  var reachedExit: Boolean = false

  var dashFrames: Int = 0
  var dashDirX: Float = 0f
  var dashDirY: Float = 0f

  val width = 48
  val height = 64

  val MAX_SPEED = 10f
  val ACCELERATION = 2f
  val GRAVITY = 0.667f
  val MAX_FALL = 26.67f
  val JUMP_VELOCITY = 15f
  val DASH_SPEED = 25f
  val DASH_DURATION = 9

  def update(level: Level): Unit = {
    reachedExit = false

    if (Gdx.input.isKeyJustPressed(Input.Keys.SHIFT_LEFT) && canDash && !isDashing) {
      startDash()
    }

    if (isDashing) {
      updateDash(level.walls, level.springs)
    } else {
      updateHorizontal()
      updateVertical()
      moveAndCollide(level.walls, level.springs)
    }

    checkLevel(level)
  }

  def updateHorizontal(): Unit = {
    val left = Gdx.input.isKeyPressed(Input.Keys.LEFT)
    val right = Gdx.input.isKeyPressed(Input.Keys.RIGHT)

    if (left && !right) {
      vx = vx - ACCELERATION
      if (vx < -MAX_SPEED) vx = -MAX_SPEED
    }
    else if (right && !left) {
      vx = vx + ACCELERATION
      if (vx > MAX_SPEED) vx = MAX_SPEED
    }
    else {
      if (vx > 0) {
        vx = vx - ACCELERATION
        if (vx < 0) vx = 0
      }
      else if (vx < 0) {
        vx = vx + ACCELERATION
        if (vx > 0) vx = 0
      }
    }
  }
}
```

```

}

def updateVertical(): Unit = {
    vy = vy - GRAVITY
    if (vy < -MAX_FALL) vy = -MAX_FALL

    if (Gdx.input.isKeyJustPressed(Input.Keys.SPACE) && onGround) {
        vy = JUMP_VELOCITY
        onGround = false
    }
}

def startDash(): Unit = {
    println("dash")
    var dx = 0f
    var dy = 0f

    if (Gdx.input.isKeyPressed(Input.Keys.LEFT)) dx = dx - 1
    if (Gdx.input.isKeyPressed(Input.Keys.RIGHT)) dx = dx + 1
    if (Gdx.input.isKeyPressed(Input.Keys.UP)) dy = dy + 1
    if (Gdx.input.isKeyPressed(Input.Keys.DOWN)) dy = dy - 1
    if (dx == 0 && dy == 0) {
        if (vx ≥ 0) dx = 1 else dx = -1
    }

    val length = Math.sqrt(dx * dx + dy * dy).toFloat
    dashDirX = dx / length
    dashDirY = dy / length

    isDashing = true
    dashFrames = DASH_DURATION
    canDash = false
}

def updateDash(walls: ArrayBuffer[Wall], springs: ArrayBuffer[Spring]): Unit = {
    vx = dashDirX * DASH_SPEED
    vy = dashDirY * DASH_SPEED

    moveAndCollide(walls, springs)
    dashFrames = dashFrames - 1

    if (dashFrames ≤ 0) {
        isDashing = false
        vx = vx / 2
        vy = vy / 2
    }
}

def moveAndCollide(walls: ArrayBuffer[Wall], springs: ArrayBuffer[Spring]): Unit = {
    x = x + vx
    for (w ← walls) {
        if (w.hits(x, y, width, height)) {
            if (vx > 0) {
                x = w.x - width
            }
            else if (vx < 0) {
                x = w.x + w.width
            }
            vx = 0
        }
    }
    for (s ← springs) {
        if (s.hits(x, y, width, height)) {
            if (vx > 0) {
                x = s.x - width
            }
            else if (vx < 0) {
                x = s.x + s.width
            }
            vx = 0
        }
    }

    onGround = false
    y = y + vy
    for (w ← walls) {
        if (w.hits(x, y, width, height)) {
            if (vy < 0) {
                y = w.y + w.height
                onGround = true
                canDash = true
            }
        }
    }
}

```

```

    }
    else if (vy > 0) {
        y = w.y - height
    }
    vy = 0
}
}
for (s ← springs) {
    if (s.hits(x, y, width, height)) {
        if (vy < 0) {
            y = s.y + s.height
            vy = s.BOUNCE_VELOCITY
            onGround = false
            canDash = true
            println("ressort")
        }
        else if (vy > 0) {
            y = s.y - height
            vy = 0
        }
    }
}
}
}

def checkLevel(level: Level): Unit = {
    for (s ← level.spikes) {
        if (s.hits(x, y, width, height)) {
            respawn()
        }
    }
    for (o ← level.orbs) {
        if (o.hits(x, y, width, height)) {
            canDash = true
            o.active = false
            o.respawnAt = System.currentTimeMillis() + o.RESPAWN_MS
            println("orbe")
        }
    }
    for (e ← level.exits) {
        if (e.hits(x, y, width, height)) {
            reachedExit = true
        }
    }
}

def respawn(): Unit = {
    println("mort")
    x = spawnX
    y = spawnY
    vx = 0
    vy = 0
    isDashing = false
    canDash = true
}

def setSpawn(sx: Float, sy: Float): Unit = {
    spawnX = sx
    spawnY = sy
    x = sx
    y = sy
}

def draw(g: GdxGraphics): Unit = {
    g.drawFilledRectangle(x + width / 2, y + height / 2, width, height, 0, Color.CYAN)
}
}

```

A.4. Level.scala

```

package ch.hevs.gdx2d.game

import ch.hevs.gdx2d.lib.GdxGraphics

import scala.collection.mutable.ArrayBuffer
import scala.io.Source

class Level(fileName: String) {

  val tileSize = 48

  val walls: ArrayBuffer[Wall] = ArrayBuffer.empty
  val spikes: ArrayBuffer[Spike] = ArrayBuffer.empty
  val exits: ArrayBuffer[Exit] = ArrayBuffer.empty
  val orbs: ArrayBuffer[Orb] = ArrayBuffer.empty
  val springs: ArrayBuffer[Spring] = ArrayBuffer.empty

  var spawnX: Float = 0
  var spawnY: Float = 0

  load(fileName)

  def load(fileName: String): Unit = {
    val src = Source.fromFile(s"levels/$fileName")
    val lines = src.getLines().toArray
    src.close()

    val nbLignes = lines.length

    for (row ← lines.indices) {
      val line = lines(row)
      for (col ← 0 until line.length) {
        val char = line.charAt(col)

        val worldX = col * tileSize
        val worldY = (nbLignes - 1 - row) * tileSize

        char match {
          case '#' ⇒ walls.addOne(new Wall(worldX, worldY, tileSize, tileSize))
          case 'X' ⇒ spikes.addOne(new Spike(worldX, worldY, tileSize, tileSize))
          case 'O' ⇒ orbs.addOne(new Orb(worldX, worldY, tileSize, tileSize))
          case 'E' ⇒ exits.addOne(new Exit(worldX, worldY, tileSize, tileSize))
          case 'S' ⇒ springs.addOne(new Spring(worldX, worldY, tileSize, tileSize))
          case 'P' ⇒
            spawnX = worldX
            spawnY = worldY
          case _ ⇒
        }
      }
    }
    println("niveau charge : " + walls.length + " murs")
  }

  def update(): Unit = {
    for (o ← orbs) {
      o.update()
    }
  }

  def draw(g: GdxGraphics): Unit = {
    for (w ← walls) w.draw(g)
    for (s ← springs) s.draw(g)
    for (spikes) s.draw(g)
    for (e ← exits) e.draw(g)
    for (o ← orbs) o.draw(g)
  }
}

```


A.5. Wall.scala

```
package ch.hevs.gdx2d.game

import ch.hevs.gdx2d.lib.GdxGraphics
import com.badlogic.gdx.graphics.Color

class Wall(val x: Float, val y: Float, val width: Float, val height: Float) {

  def draw(g: GdxGraphics): Unit = {
    g.drawFilledRectangle(x + width / 2, y + height / 2, width, height, 0, Color.GRAY)
  }

  def hits(px: Float, py: Float, pw: Float, ph: Float): Boolean = {
    px < x + width && px + pw > x && py < y + height && py + ph > y
  }
}
```

A.6. Spike.scala

```
package ch.hevs.gdx2d.game

import ch.hevs.gdx2d.lib.GdxGraphics
import com.badlogic.gdx.graphics.Color

class Spike(val x: Float, val y: Float, val width: Float, val height: Float) {

  def draw(g: GdxGraphics): Unit = {
    g.drawFilledRectangle(x + width / 2, y + height / 2, width, height, 0, Color.RED)
  }

  def hits(px: Float, py: Float, pw: Float, ph: Float): Boolean = {
    px < x + width && px + pw > x && py < y + height && py + ph > y
  }
}
```

A.7. Spring.scala

```
package ch.hevs.gdx2d.game

import ch.hevs.gdx2d.lib.GdxGraphics
import com.badlogic.gdx.graphics.Color

class Spring(val x: Float, val y: Float, val width: Float, val height: Float) {

    val BOUNCE_VELOCITY = 24f

    def draw(g: GdxGraphics): Unit = {
        g.drawFilledRectangle(x + width / 2, y + height / 2, width, height, 0, Color.BROWN)
    }

    def hits(px: Float, py: Float, pw: Float, ph: Float): Boolean = {
        px < x + width && px + pw > x && py < y + height && py + ph > y
    }
}
```

A.8. Orb.scala

```
package ch.hevs.gdx2d.game

import ch.hevs.gdx2d.lib.GdxGraphics
import com.badlogic.gdx.graphics.Color

class Orb(val x: Float, val y: Float, val width: Float, val height: Float) {

    var active: Boolean = true
    var respawnAt: Long = 0

    val RESPAWN_MS = 3000

    def update(): Unit = {
        if (!active && System.currentTimeMillis() ≥ respawnAt) {
            active = true
        }
    }

    def hits(px: Float, py: Float, pw: Float, ph: Float): Boolean = {
        active && px < x + width && px + pw > x && py < y + height && py + ph > y
    }

    def draw(g: GdxGraphics): Unit = {
        if (active) {
            g.drawFilledCircle(x + width / 2, y + height / 2, 14, Color.GREEN)
        }
    }
}
```

A.9. Exit.scala

```
package ch.hevs.gdx2d.game

import ch.hevs.gdx2d.lib.GdxGraphics
import com.badlogic.gdx.graphics.Color

class Exit(val x: Float, val y: Float, val width: Float, val height: Float) {

  def draw(g: GdxGraphics): Unit = {
    g.drawFilledRectangle(x + width / 2, y + height / 2, width, height, 0, Color.YELLOW)
  }

  def hits(px: Float, py: Float, pw: Float, ph: Float): Boolean = {
    px < x + width && px + pw > x && py < y + height && py + ph > y
  }
}
```